

COMP90041

Programming and Software Development

Lecture 9 - Exceptions and File Handling

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968* (**Act**).

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Overview



In this part of the lecture, you will learn about:

- **Exceptions & Exceptions Handling**
- **Files I/O**



THE UNIVERSITY OF
MELBOURNE

Exceptions

What are Exceptions?

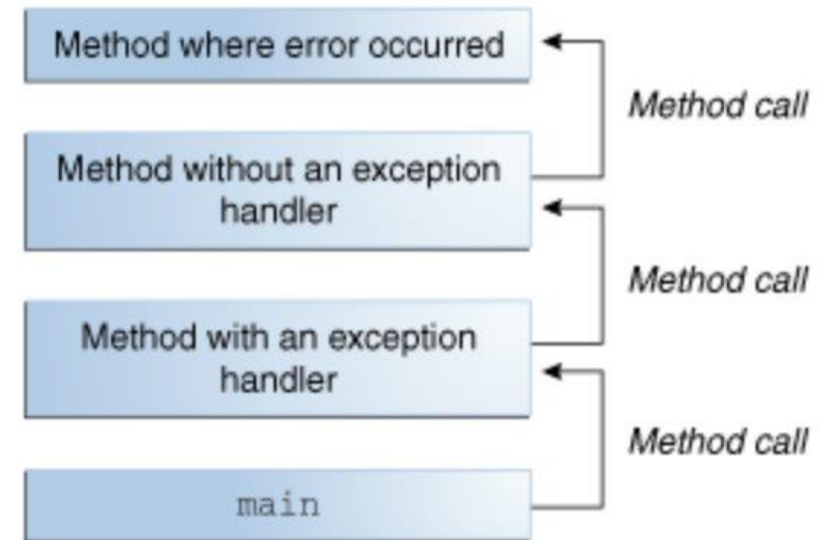


- An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- Java Runtime signals the operating system something unusual has happened by **throwing** an exception.
- If you do not catch an exception, the operating system stops the program execution.
- Programmers must handle the unusual behaviour by **catching** the exception. In this case, operating system passes the control back to the program.

Exceptions Stack



- When exception occurs, it creates an entire stack of messages that contains lines of methods, starting from the method that was called upto the method where the exception occurred.
- Exceptions thrown by the runtime has two parts
 - Messages – actual error that occurred
 - Stack Message – the entire stack of methods and Line numbers where the exception has happened.



Try-catch blocks



- To handle an exception we can use try-catch blocks
- Wrap the code that can cause an exception in a try block. If everything goes smoothly, try block executes normally and program execution goes to the next line after the catch.
- If an exception happens, the program execution stops at the line where exception has happened and control is passed to the catch block.
- When **an exception happens you can**
 - Either quit the program gracefully by printing an error message
 - Can prompt the user to take an appropriate action such as reentering inputs
 - Can lead to an alternate flow of a program by invoking different method calls or setting default values

Try-catch blocks



Input mismatch
exception

Divide by zero

```
public void someMethod{
    try{
        int a = 13;
        Scanner input = new Scanner(System.in);
        int b = input.nextInt();
        System.out.printf("Division %d/%d = %d", a, b, a/b);
    }catch(Exception ex){
        System.out.println("Some exception happened: " + ex.getMessage());
        ex.printStackTrace();
    }
    System.out.println("Other statements can go here");
}
```

One exception
handles all

Nested catch blocks



```
public void someMethod{
    Scanner input = new Scanner(System.in);
    int a = 13;
    boolean invalidInput = true;
    while(invalidInput){
        try{
            int b = input.nextInt();
            invalidInput = false;
            System.out.printf("Division %d/%d = %d", a, b, a/b);
        }catch(InputMismatchException ex){
            System.out.println("Input should be an integer. Enter again:");
        }catch(ArithmeticException ex){
            System.out.println("Divide by zero, quitting program");
            return;
        }catch(Exception ex){
            System.out.println("Some other exception happened");
        }
        //continue other java statements after catch blocks
    }
}
```

Input mismatch exception,
invalidInput is still true, loops
allows input again

Divide by zero, print message
and return from the method

Pitfalls: Nested catch blocks



```
public void someMethod(){
    Scanner input = new Scanner(System.in);
    int a = 13;
    boolean invalidInput = true;
    while(invalidInput){
        try{
            int b = input.nextInt();
            invalidInput = false;
            System.out.printf("Division %d/%d = %d", a, b, a/b);
        }catch(Exception ex){
            System.out.println("Some other exception happened");
        }catch(InputMismatchException ex){
            System.out.println("Input should be an integer. Enter again:");
        }catch(ArithmeticException ex){
            System.out.println("Divide by zero, quitting program");
            return;
        }
        //continue other java statements after catch blocks
    }
}
```

Whichever catch block is matched first, the control is passed to that block. Other blocks are not executed. Exception is super class to every exception class. Put subclasses first in the hierarchy.

Java Exception Classes



- Java has many Exception classes. Some of the famous ones -
- RuntimeException – anything that happened during runtime. Subclasses -
 - ArithmeticException – divide by zero
 - ClassCastException – Remember inheritance upcasting/downcasting?
 - NullPointerException – accessing variables or methods of variables that are null.
 - NoSuchElementException – issues with your Scanner object taking inputs? Input is not found?
 - IndexOutOfBoundsException (or ArrayIndexOutOfBoundsException) - accessing index that does is greater than the size of the array.
- IOException – exceptions accessing input/output like files. Subclasses -
 - FileNotFoundException – file is not present in the filesystem or cannot be opened for read/write.
 - EOFException – end of file is met while reading input
- Many more Exception classes present in JAVA. Check [Javadocs](#).

Defining Custom Exception Class



- You can define your own custom Exception class.
- To define custom Exception classes, extend from the Exception class.
- Good practice – define a constructor that take the message as parameters. Do not hardcode an error message in the constructor.

```
public class MathException extends Exception{  
  
    public MathException(){}  
  
    public MathException(String message){  
        super(message);  
    }  
}
```

- Reuse the same constructor to use different messages like `new MathException("DivideByZero happened")` or `new MathException("Arithmetic Overflow happened")`;

Throw vs Throws



- Once you create a custom Exception class, you can **throw** them from a method.
- For ex – incorrect password – **throw new** IncorrectPasswordException("Password is missing"); Or **throw new** IncorrectPasswordException("Password does not match");

```
public boolean VerifyPassword(String inputPwd, String actualPwd) throws
IncorrectPasswordException{
    if(inputPwd == null || inputPwd.equals("")){
        throw new IncorrectPasswordException("Password is missing");
    }else if(! inputPwd.equalsIgnoreCase(actualPwd)){
        throw new IncorrectPasswordException("Password does not match");
    }
}
```

Throws
keyword
here in
method
definition

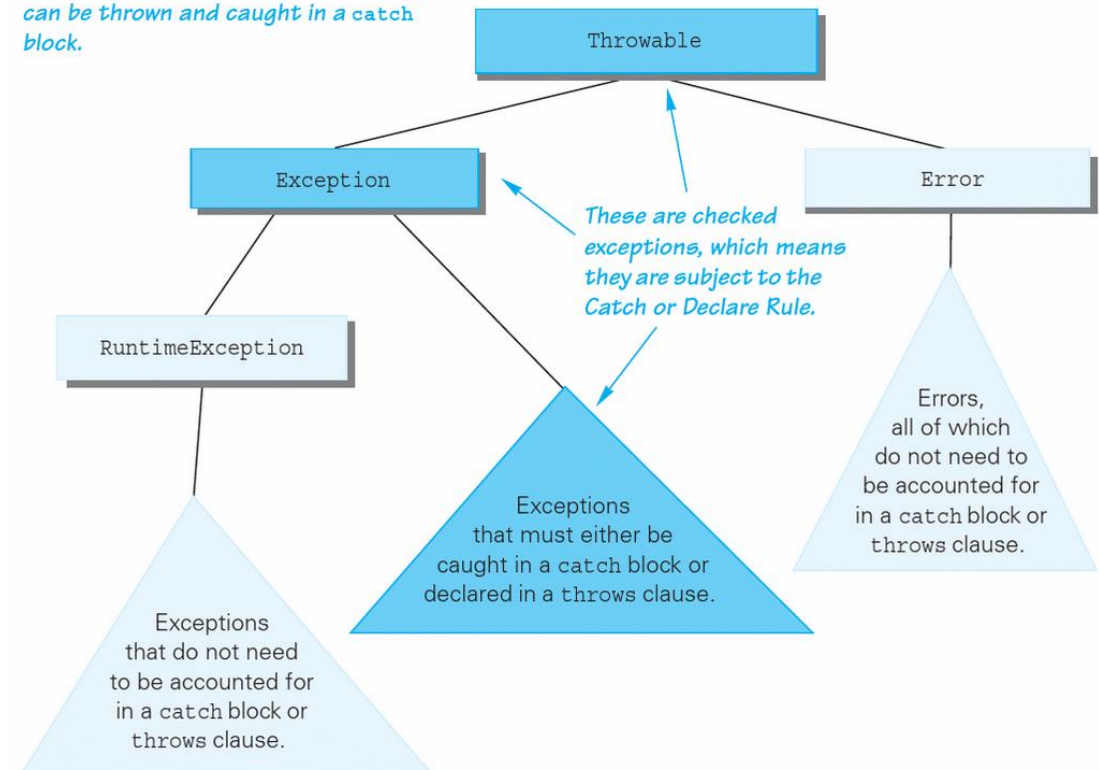
- When your method throws an exception but not handle it using a catch block, the method definition must define what exception the method can throw using **throws** keyword.

Advanced Topics: Throwable

- Throwable is a class.
- Both Exception and Error extend class Throwable.
- Errors are something that the code should not try to catch.
- When you throw an exception or error, you tell the Operating System (OS) that something is wrong with a positive or negative integer code.
- Example – if you can't find a file to write, you can always create it. What if you want to print something and you can't find the printer? Throw the error, the program terminates, OS receives a negative signal, logs the error in Operating System logs.

Display 9.11 Hierarchy of Throwable Objects

All descendents of the class Throwable can be thrown and caught in a catch block.



Finally block



- An Exception stops the current execution and resumes from the catch block.
- In some scenarios we would like the program to perform special actions whether or not the exception has happened.
- For these actions/tasks, you can put the code blocks in finally.
- Example – opening a file for reading/writing, half way through the file is opened, an exception occurs. You must close the file. Put it in the finally block.
- A finally will execute even when an exception has occurred.

```
try{
    // critical code that can cause exception goes here
}catch(Exception ex){
    //handle a series of exceptions here by creating nested exceptions blocks
}finally{
    // do some critical tasks here like file closing or closing Scanner objects
}
```



THE UNIVERSITY OF
MELBOURNE

Live Coding (Exceptions)

Exceptions



- **Demonstrate try-catch-finally block with system exceptions**
 - **NumberFormatException**
 - **NullPointerException**
 - **InputMismatchException**
- **Create a custom Exception class**
 - **Throw custom exception with message**
 - **Catch and handle custom exception**



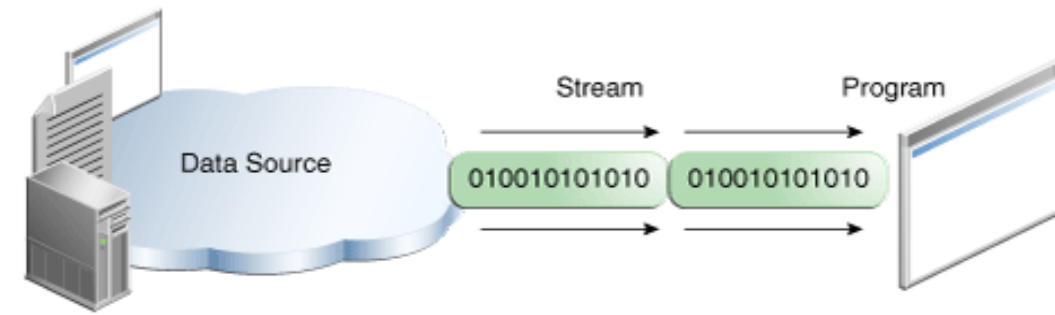
THE UNIVERSITY OF
MELBOURNE

Files I/O

File Streams



- Why files?
 - Because a program hold data in the memory. When the program terminates, the memory is released.
 - Large data that cannot stay in the memory can be stored in files as chunks.
 - Data saved by one program can be used by other programs.
- How files are used?
 - Files are used as streams of data or a sequence of bytes one after other.
 - Data is read from an *input stream*.
 - Data written into a file uses an *output stream*.



File Streams



- Two kinds of files
 - Text Files – files containing numbers, Strings and other readable data format.
 - Binary Files – audio/video files, images, etc.
- You can read/write the files using System streams
 - Output stream – `System.out`
 - Input stream – `System.in`

Reading files: Scanner



- Use Scanner to read any file including Standard input.
- Scanner constructor takes a FileInputStream
- System.in returns FileInputStream
- To read a file pass a new FileInputStream(filename) as an argument to Scanner

```
Scanner scanner = new Scanner(System.in); // reading from System input
int a = scanner.nextInt();
```

```
Scanner scanner = new Scanner(new FileInputStream("C://input.txt"));
int a = scanner.nextInt(); //reads first integer
float b = scanner.nextFloat(); // reads the other float
scanner.nextLine(); // goes to the next line
```

```
//repeat reading other lines and inputs on other lines one by one.
```

Look at
nested
constructors

Reading files: Scanner



- When do you stop making `scanner.nextLine()` statements or how do you know when there is no more input present in the file?
- A better way to read is parse the file line by line.
- Use the `hasNextLine()` method to see if the end of file has been reached.
- Don't forget to close the scanner.
- Exceptions:
 - `FileNotFoundException` if the file does not exist or is not readable.
 - `NoSuchElementException` if there are no more lines.
 - `IllegalStateException` if the Scanner stream is closed

Reading files: Scanner



```
Scanner input= null;
try{
    input = new Scanner(new FileInputStream("C:\\input.txt"));
    while(input.hasNextLine()){
        String text = input.nextLine(); // reads the line
        // you can split the line based on space and read the inputs
        String[] parts = text.split(" ");
        //use the parts from split to perform any actions
    }
}catch(Exception ex){
    //handle each exception differently in nested catch blocks
}finally{
    if(input != null)
        input.close(); // don't forget to close the scanner in finally
}
```

File Writing: PrintWriter



- Easy writing into the files using `PrintWriter`.
- Similar to `Scanner` takes a new `FileOutputStream(fileName)`;
- Use the `writeInt`, `writeChars` or `writeDouble` methods to convert and write `int`, `String`, `double` into bytes
- Remember to close the printer.
- Exceptions :
 - `FileOutputStream` throws a `FileNotFoundException` if file is not ready for writing.

```
PrintWriter printer= null;
try{
    printer= new PrintWriter(new FileOutputStream("C:\\output.txt"));
    printer.print("This is a text");
    printer.println("This is a line");
}catch(Exception ex){
    // handle Exceptions
}finally{
    if (printer != null) // good practice
        printer.close(); // don't forget to close the printWriter in finally
}
```

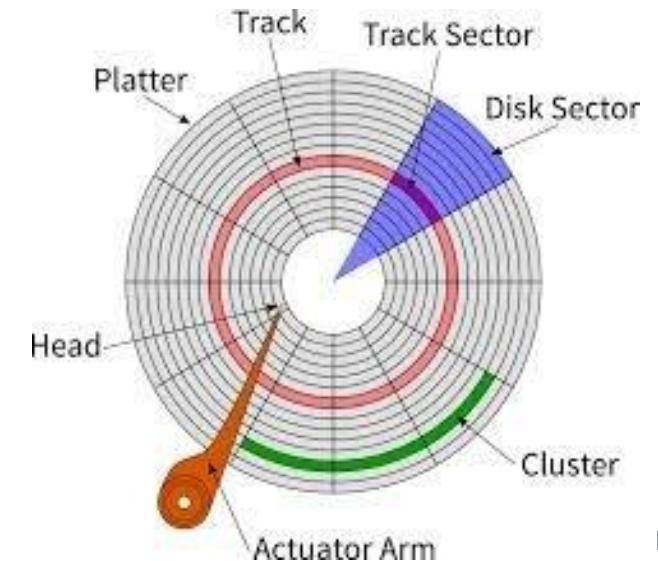
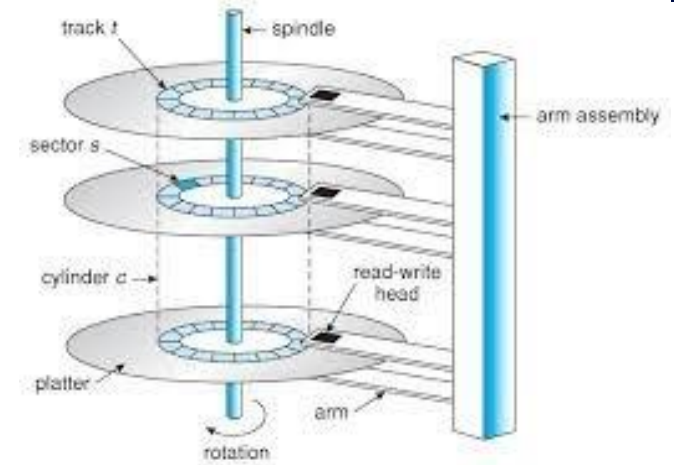
Writing vs Appending



- While writing to a file, when a filename is passed, JAVA libraries look for the file at the path.
 - If the file is not found, then a new file will be created. No exceptions thrown.
 - If the file is found, but it is not writeable due to some reason(locked by other software), system may throw a `FileNotFoundException`.
- Once you start writing to the file using `PrintWriter` and `FileOutputStream`, everything is written to the file. Default mode is “overwrite” mode.
- If you want to append to the content of the file, use another param while creating `FileOutputStream` — `new FileOutputStream(filename, true)`; The `true` ensures that the file is opened in append mode.

Disk Reading/Writing (Non Examinable)

- Disk read/writes are heavy operation.
 - Physical operation requires an arm to select the right disk first
 - Then move on to the correct disk sector
 - Then select the right track.
 - Lots of physical arm movement to reach the data.
 - Harder to read data if they are on different sectors.
 - Additional issues, if the file size is really really large.
-
- We expect while reading/writing a file, the memory is consecutive and present in same/adjacent sectors.
 - Read/Write bulk of them but the program doesn't need everything. Solution – buffers
 - High probability that adjacent data will be needed soon.



Buffered Reading/Writing



- Buffers are temporary short memory blocks that OS uses.
- While reading,
 - additional data from the hard disk is brought into buffer. But only required data is returned to the program.
 - When next data is requested by the program, it is first searched in the buffer, if not found, OS goes back to hard disk to read the data.
 - If found in buffer, return from the buffer.
- While writing
 - Write the data first into the buffer.
 - When enough data is accumulated or a set amount of time is expired, ***flush*** the data onto hard disk.

Buffered Reading

- Use `FileReader` and `BufferedReader` instead of `FileInputStream` and `Scanner` respectively.

```
BufferedReader br = null;
FileReader fr = null;
try{
    fr = new FileReader("data.txt"); //relative path
    br = new BufferedReader(fr);
    String line;
    while ((line = br.readLine()) != null) { // checks end of file
        System.out.println(line);
    }
}catch(IOException ex){
    //handle exceptions as per requirement
}finally{
    if(br != null) // good practice
        br.close(); // look at the order of the close
    if(fr != null) // good practice
        fr.close();
}
```

Buffered Writing



Use `FileWriter` and `BufferedWriter` instead of `FileOutputStream` and `PrintWriter` respectively.

```
BufferedWriter bw= null;
FileWriter fw = null;
String filePath = "/users/trinadey/downloads/data.txt";//linux style absolute path
try {
    fw = new FileWriter(filePath);
    bw = new BufferedWriter(fw);
    bw.write("This is the first line.");
    bw.newLine(); // Adds a newline
    // BufferedWriter holds the data in memory until buffer is full or flushed
    bw.flush(); // Optional, ensures data is written immediately once you are done
} catch (IOException ex) {
    ex.printStackTrace();
}finally{
    if(bw != null) // good practice
        bw.close();
    if(fw!= null) // good practice
        fw.close();
}
```

Other Java Readers



Class	Description
Reader	Abstract base class for all character input streams.
InputStreamReader	Converts byte stream (e.g. from file or network) into a character stream using a specified charset.
FileReader	Reads character files (uses platform's default charset).
StringReader	Reads from a string as if it's a stream.
CharArrayReader	Reads characters from a char array.
BufferedReader	Buffers characters for efficient reading, includes <code>readLine()</code> .
LineNumberReader	Extends <code>BufferedReader</code> , keeps track of line numbers.
PushbackReader	Allows "unreading" characters (push them back to the stream).
PipedReader	Used for thread-to-thread communication (with <code>PipedWriter</code>).

Other Java Writers



Class	Description
Writer	Abstract base class for all character output streams.
OutputStreamWriter	Converts character stream into a byte stream using a specified charset.
FileWriter	Writes character files (uses platform's default charset).
StringWriter	Writes to a string buffer (in-memory).
CharArrayWriter	Writes characters to a char array.
BufferedWriter	Buffers characters for efficient writing, includes <code>newLine()</code> .
PrintWriter	Provides convenient methods like <code>print()</code> , <code>println()</code> , supports formatting.
PipedWriter	Used for thread-to-thread communication (with <code>PipedReader</code>).

File Paths



- If the file path starts with a / or a drive name like C:\\, Java Runtime starts searching from the **absolute path** provided.
- If the filepath starts with a folder name or a file name directly, Java Runtime will start looking from the CLASSPATH (or current running directory i.e where the main method is)
- You can also use relative path to other directories by using
 - ./ defines current directory
 - ../ will define parent directory
 - Use as many ../ as you want to move to the parent of parent directory and so on
- Remember that \ is also used as an escape character. So if you are using Windows, escape the \ in the filepaths like C:\\\\somefolder\\somefile.txt

File Class



- The File class is a wrapper for filenames.
- `new Scanner(new FileInputStream("C://input.txt"))`; can also be written as `new Scanner(new FileInputStream(new File("C://input.txt")))`
- Why would we do this? Why complicate life?
 - You can use methods from File class to check if the file exists or the folder exists
 - While FileOutputStream will automatically create the filename, it won't create a folder.
 - You can create folders using mkdir or mkdirs(creates all the nested parent folders) methods.
 - You can also set the files as readonly after you have written the file.
 - And many more. Checkout EdStem [here](#).



THE UNIVERSITY OF
MELBOURNE

Live Coding

Live Coding – Reading & Writing Files!!



- Read from one file and write into another file.
- Create folders while writing files.
- See BufferedWriters don't flush easily.

Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 9)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 10)
- Java Exceptions (accessible on 14-02-2024) Oracle's Java Documentation. Available at: <https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>.
- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 10)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 22)

Copyright Notice



(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: copyright-office@unimelb.edu.au

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!



THE UNIVERSITY OF
MELBOURNE